

CI/CD Pipelines

Complete Interview Preparation Guide · 60 Questions

60

Questions

9

Sections

Jenkins

GitHub Actions

Senior

Level

Jenkins · GitHub Actions · GitLab CI/CD · Pipeline Design · Docker & Kubernetes in CI/CD · Testing · Security & Secrets · GitOps & ArgoCD · Real-World Scenarios

Sandhya

DevOps Engineer · Birlasoft

© Sandhya · DevOps Interview Mastery Series

SECTION 01

CI/CD Fundamentals

Core concepts, pipeline stages, triggers, artifacts

Q01

What is CI/CD and what problem does it solve in software delivery?

• Easy

ANSWER

CI/CD stands for Continuous Integration and Continuous Delivery/Deployment. It is a set of practices and automation tools that enable development teams to deliver software changes faster, more reliably, and with less manual effort.

The problem it solves: Before CI/CD, developers would work in isolation for weeks or months, then attempt to merge massive changes — leading to integration hell, broken builds, delayed releases, and high-risk manual deployments.

CI/CD Stage	What It Does	Problem It Solves
Continuous Integration	Auto-build + test on every commit	Catch bugs early, not weeks later
Continuous Delivery	Auto-deploy to staging, manual prod trigger	Reliable, repeatable deployments
Continuous Deployment	Auto-deploy to production on every passing build	Zero manual release process

Key benefits: faster feedback loops, reduced integration risk, consistent deployments, auditable change history, and the ability to release multiple times per day safely.

📌 Interviewers often ask 'Delivery vs Deployment' — Continuous Delivery means production deployment is a manual decision. Continuous Deployment means every passing build automatically goes to production. Most enterprises use Delivery, not Deployment.

KEY TAKEAWAYS

- ▶ CI = build + test on every commit; CD = automated deployment pipeline
- ▶ Delivery = manual prod trigger; Deployment = fully automated to production
- ▶ CI/CD eliminates integration hell and makes releases boring (predictable)

Q02

What is the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment?

• Easy

ANSWER

Concept	Definition	Automation Level	Human Trigger Needed
Continuous Integration	Every commit triggers build + tests	Build, unit tests, static analysis	No — fully automatic
Continuous Delivery	Code is always in a deployable state; deploy to staging automatically	CI + integration tests + staging deploy	Yes — production deploy is manual

Continuous Deployment

Every passing build deploys to production automatically

Everything including production

No — fully automatic end-to-end

Most mature organizations practice Continuous Delivery — they automate everything but keep a manual approval gate before production. Continuous Deployment is used by companies like Netflix, Amazon, and Etsy that have extremely high test coverage and confidence.

KEY TAKEAWAYS

- ▶ CI stops at test results; CD extends to deployment
- ▶ Continuous Delivery = deploy to staging automatically, production manually
- ▶ Continuous Deployment = production fully automated — requires very high test confidence

Q03

What are the core stages of a CI/CD pipeline — explain each stage?

• Easy

ANSWER

Stage	Purpose	Tools
Source / Trigger	Detect code change, start pipeline	Git, GitHub, GitLab, Bitbucket
Build	Compile code, create artifact	Maven, Gradle, npm, Docker build
Unit Test	Fast isolated tests — no external deps	JUnit, pytest, Jest, Go test
Static Analysis	Code quality, security scan, linting	SonarQube, ESLint, Checkstyle
Integration Test	Test with real DBs, services, APIs	Testcontainers, Postman, REST Assured
Package / Publish	Create final artifact, push to registry	Docker Hub, ECR, Nexus, Artifactory
Deploy to Staging	Deploy to staging environment	Helm, kubectl, Ansible, Terraform
Smoke / E2E Test	Validate staging deployment works	Selenium, Cypress, k6
Deploy to Production	Release to live environment	ArgoCD, Helm, Spinnaker, kubectl
Monitor	Verify production health post-deploy	Prometheus, Grafana, Datadog

🔗 Not every pipeline has all 10 stages. A good answer shows you understand the progression: fast tests first (fail fast), slow tests later, deploy only after all gates pass.

KEY TAKEAWAYS

- ▶ Always: Source → Build → Test → Package → Deploy → Verify
- ▶ Fail fast principle: run fastest tests first to give quick feedback
- ▶ Never deploy an artifact that failed any stage — the artifact is the source of truth

Q04

What is a pipeline trigger — what events can trigger a CI/CD pipeline?

• Easy

ANSWER

A pipeline trigger is an event that causes a CI/CD pipeline to start automatically. Different trigger types serve different purposes in the development workflow.

Trigger Type	Event	Use Case
Push trigger	Code pushed to any branch	Run CI on every developer commit
Pull Request trigger	PR opened or updated	Validate code before merge
Merge trigger	PR merged to main/master	Deploy to staging after merge
Tag trigger	Git tag created (e.g. v1.2.0)	Production release pipeline
Schedule trigger	Cron expression (e.g. nightly)	Nightly builds, regression tests
Manual trigger	Developer clicks 'Run'	Production deployment approval
Webhook trigger	External system event	Cross-repo dependencies, Jira integration
API trigger	HTTP call to pipeline API	Programmatic pipeline execution

KEY TAKEAWAYS

- ▶ Push = CI on every commit; Tag = production release; Schedule = nightly runs
- ▶ PR trigger is critical — validate code BEFORE it merges to main
- ▶ Manual trigger for production gives the human approval gate in Continuous Delivery

Q05**What is the difference between a build artifact and a deployment artifact?**

● Medium

ANSWER

Build Artifact — the output of the build stage: compiled code, JAR, WAR, binary, npm bundle. It is environment-agnostic — the same build artifact should be deployable to any environment.

Deployment Artifact — the package used to deploy the application to a specific environment. It combines the build artifact with environment-specific configuration (env vars, secrets, connection strings).

```
// Artifact flow example
// Build stage — produces environment-agnostic artifact
docker build -t myapp:1.2.3 .
docker push registry.io/myapp:1.2.3 ← build artifact (image)

// Deploy to staging — deployment artifact = image + staging config
helm upgrade myapp ./chart \
  --set image.tag=1.2.3 \
  --set env=staging \
  --values staging-values.yaml

// Deploy to production — same image, different config
helm upgrade myapp ./chart \
  --set image.tag=1.2.3 \
  --set env=production \
```

```
--values prod-values.yaml
```

Key principle: Build once, deploy anywhere. The same Docker image or JAR should go through staging and then to production — never rebuild for production. Rebuilding risks environment differences.

🔗 'Build once, deploy anywhere' is a core CI/CD principle. If you rebuild for production, you cannot guarantee that production runs the same code that passed staging tests.

KEY TAKEAWAYS

- ▶ Build artifact = environment-agnostic output (Docker image, JAR, binary)
- ▶ Deployment artifact = build artifact + environment config combined
- ▶ Never rebuild for production — the artifact that passed staging IS the production artifact

Q06

What is a pipeline as code and why is it better than GUI-configured pipelines?

● Medium

ANSWER

Pipeline as Code means defining your CI/CD pipeline in a text file (Jenkinsfile, .github/workflows/ci.yml, .gitlab-ci.yml) stored in your version control repository alongside the application code.

Aspect	GUI-Configured Pipeline	Pipeline as Code
Version Control	Not versioned — changes are invisible	Full Git history — every change tracked
Review Process	No peer review possible	PR-reviewed like application code
Reproducibility	Hard to recreate if server lost	Recreate instantly from repo
Reuse	Copy-paste via UI, error-prone	Shared libraries, templates, includes
Audit Trail	Limited logging	Git blame shows who changed what and why
Disaster Recovery	Manual reconfiguration	Restore by cloning repo

Pipeline as Code is an extension of Infrastructure as Code — treating your CI/CD configuration with the same discipline as application code.

KEY TAKEAWAYS

- ▶ Pipeline as Code = Jenkinsfile/.yml stored in Git with the application
- ▶ Changes are reviewed, versioned, and auditable — same as application code
- ▶ GUI pipelines are lost if the CI server is rebuilt; Code pipelines are reproduced instantly

SECTION 02

Jenkins

Architecture, Pipelines, Shared Libraries, credentials, debugging

Q07

What is Jenkins and what is its role in a CI/CD pipeline?

• Easy

ANSWER

Jenkins is an open-source automation server written in Java that orchestrates CI/CD pipelines. It is the most widely deployed CI/CD tool in enterprise environments, with over 1,800 plugins covering integration with virtually every DevOps tool.

Jenkins role in CI/CD:

- Listens for triggers (Git webhooks, schedules, manual)
- Allocates agent nodes to run pipeline stages
- Executes build, test, and deploy steps in defined order
- Reports results — pass/fail, test reports, logs
- Manages credentials securely for external systems
- Archives artifacts and triggers downstream pipelines

Component	Role
Jenkins Controller (Master)	Schedules jobs, manages agents, stores config, serves UI
Jenkins Agent (Node)	Executes pipeline steps — does the actual work
Jenkinsfile	Pipeline definition stored in Git
Plugin	Extends Jenkins — Git, Docker, Kubernetes, Slack plugins
Credentials Store	Secure storage for passwords, tokens, SSH keys

KEY TAKEAWAYS

- Jenkins = automation server that orchestrates CI/CD workflows
- Controller schedules; Agents execute — never run builds on the controller in production
- 1800+ plugins = Jenkins integrates with almost everything

Q08

What is the difference between a Freestyle Job and a Pipeline Job in Jenkins?

• Medium

ANSWER

Aspect	Freestyle Job	Pipeline Job
Definition	GUI-configured job with build steps	Code-defined pipeline in Jenkinsfile

Version Control	Not stored in Git	Jenkinsfile committed to repo
Complexity	Simple, single-step tasks	Multi-stage, parallel, conditional logic
Reusability	No — must reconfigure per job	Yes — Shared Libraries, templates
Parallel Steps	Not supported	Native parallel blocks
Audit Trail	No — config in Jenkins UI	Full Git history
Recovery	Manual UI reconfiguration	Auto-restored from Jenkinsfile
Recommended For	Quick experiments, legacy	All new pipelines — standard

Freestyle jobs were the original Jenkins job type. Pipeline jobs (using Jenkinsfile) are the modern standard and should be used for all new pipelines.

KEY TAKEAWAYS

- Freestyle = GUI config, no version control; Pipeline = Jenkinsfile in Git
- Pipeline jobs support parallel stages, conditions, retry, and complex logic
- Never use Freestyle for new pipelines — Pipeline jobs are the standard

Q09

What is a Jenkinsfile and what are the two pipeline syntaxes — Declarative vs Scripted?

● Medium

ANSWER

A Jenkinsfile is a text file that defines a Jenkins pipeline using either Declarative or Scripted syntax. It lives in the root of the Git repository and is checked out automatically by Jenkins.

```
// Declarative Pipeline (recommended)
pipeline {
  agent { label 'linux' }
  environment {
    APP_VERSION = '1.2.3'
    DOCKER_REGISTRY = 'registry.io'
  }
  stages {
    stage('Build') {
      steps {
        sh 'mvn clean package -DskipTests'
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test'
      }
      post { always { junit 'target/surefire-reports/*.xml' } }
    }
    stage('Docker Build') {
      steps {
        sh 'docker build -t $DOCKER_REGISTRY/myapp:$APP_VERSION .'
      }
    }
  }
}
```

```

    }
  }
  post {
    failure { slackSend message: 'Build FAILED: ${env.JOB_NAME}' }
    success { slackSend message: 'Build PASSED: ${env.JOB_NAME}' }
  }
}

```

Aspect	Declarative	Scripted
Syntax	Structured, opinionated DSL	Groovy code — full flexibility
Learning Curve	Low — readable by non-developers	High — requires Groovy knowledge
Error Messages	Clear, specific	Generic Groovy errors
Validation	Pre-execution syntax check	Fails at runtime
Recommended	Yes — for almost all pipelines	Only for very complex custom logic

📌 Always recommend Declarative in interviews. Scripted gives more power but Declarative is readable, validated, and what 95% of teams use. Knowing both shows depth.

KEY TAKEAWAYS

- ▶ Declarative = structured DSL with pipeline{} wrapper — recommended for all new pipelines
- ▶ Scripted = Groovy code — more flexible but harder to read and debug
- ▶ Jenkinsfile lives in Git root — Jenkins checks it out and runs it automatically

Q10
What is a Jenkins Agent and what is the difference between a Master and Agent node?

● Medium

ANSWER

Jenkins uses a Master-Agent (Controller-Agent) architecture to distribute pipeline execution across multiple machines.

Aspect	Jenkins Controller (Master)	Jenkins Agent (Node)
Role	Orchestrates, schedules, manages state	Executes actual pipeline steps
Build Execution	Should NEVER run builds in production	Runs all build/test/deploy steps
Resources	Needs minimal CPU — management only	Needs build tools, Docker, JDK etc.

Scaling	Single controller (or HA pair)	Scale out — add agents as load grows
Types	Fixed — always running	Static (persistent) or Dynamic (ephemeral)
Example	Jenkins server on EC2 t3.medium	Build agents on EC2 c5.2xlarge

```
// Agent definition in Jenkinsfile
// Run on any available agent
agent any

// Run on specific labeled agent
agent { label 'docker-agent' }

// Run in Docker container (ephemeral agent)
agent {
    docker {
        image 'maven:3.9-openjdk-17'
        args '-v /root/.m2:/root/.m2'
    }
}

// Kubernetes pod agent (dynamic)
agent {
    kubernetes {
        yaml '''
spec:
  containers:
  - name: maven
    image: maven:3.9-openjdk-17
'''
    }
}
```

KEY TAKEAWAYS

- ▶ Controller orchestrates; Agents execute — separate concerns
- ▶ Never run builds on Controller in production — security and stability risk
- ▶ Dynamic Kubernetes agents spin up as pods per build and terminate after — elastic scaling

Q11**What are Jenkins Shared Libraries and why are they used in large organizations?**

● Hard

ANSWER

Jenkins Shared Libraries are reusable Groovy code stored in a separate Git repository that multiple Jenkins pipelines can import. They allow organizations to standardize pipeline logic, enforce policies, and eliminate copy-paste duplication across hundreds of Jenkinsfiles.

```
// Shared Library structure
jenkins-shared-library/
```

```
vars/
  buildDockerImage.groovy    ← global variable (callable as function)
  deployToKubernetes.groovy
  runMavenBuild.groovy
src/
  org/company/
    PipelineUtils.groovy    ← Groovy classes
resources/
  docker/Dockerfile.template

// Using Shared Library in Jenkinsfile
// Import the shared library (configured in Jenkins global settings)
@Library('company-jenkins-lib@main') _

pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        // Call shared library function
        runMavenBuild(goals: 'clean package', skipTests: false)
      }
    }
    stage('Docker') {
      steps {
        buildDockerImage(name: 'myapp', tag: env.BUILD_NUMBER)
      }
    }
    stage('Deploy') {
      steps {
        deployToKubernetes(env: 'staging', image: 'myapp:'+env.BUILD_NUMBER)
      }
    }
  }
}
```

Benefits: one change to the shared library updates all pipelines; security and compliance logic can be centrally enforced; teams only write business-specific steps.

KEY TAKEAWAYS

- ▶ Shared Libraries = reusable pipeline code in separate Git repo, imported by Jenkinsfiles
- ▶ vars/ folder = callable functions; src/ folder = Groovy classes
- ▶ One library update propagates to all pipelines — centralized governance

Q12**What is a Jenkins Multibranch Pipeline and how does it work with Git branches?**

● Medium

ANSWER

A Multibranch Pipeline is a special Jenkins job type that automatically discovers branches and Pull Requests in a Git repository and creates a separate pipeline for each one. The pipeline is defined by the Jenkinsfile in each branch.

How it works:

- ▶ Jenkins scans the repository periodically (or via webhook)

- ▶ For every branch containing a Jenkinsfile, Jenkins creates a sub-pipeline
- ▶ PRs get their own isolated pipeline — merge validation before touching main
- ▶ Branches that are deleted automatically have their Jenkins pipeline removed

Branch Pattern	Typical Pipeline Behavior
main / master	Full pipeline: build + test + deploy to production
develop	Build + test + deploy to staging
feature/*	Build + unit tests only (fast feedback)
release/*	Build + full test suite + staging deploy
PR branches	Build + test + post results to PR — no deploy

Multibranch is the recommended Jenkins setup for modern Git workflows (GitFlow, GitHub Flow, trunk-based development).

KEY TAKEAWAYS

- ▶ Multibranch = one Jenkins job auto-creates pipelines for every branch with a Jenkinsfile
- ▶ Each branch gets its own isolated build history and pipeline behavior
- ▶ PR pipelines validate code before merge — essential for trunk-based development

Q13

How do you securely manage secrets and credentials in Jenkins?

● Hard

ANSWER

Jenkins has a built-in Credentials Store for securely storing and injecting secrets into pipelines. Secrets are never stored in Jenkinsfiles or Git.

```
// Credential types and usage
// 1. Username + Password
withCredentials([usernamePassword(
    credentialsId: 'docker-hub-creds',
    usernameVariable: 'DOCKER_USER',
    passwordVariable: 'DOCKER_PASS'
)]) {
    sh 'docker login -u $DOCKER_USER -p $DOCKER_PASS'
}

// 2. Secret Text (API token, connection string)
withCredentials([string(credentialsId: 'sonar-token', variable: 'SONAR_TOKEN')]) {
    sh 'sonar-scanner -Dsonar.login=$SONAR_TOKEN'
}

// 3. SSH Private Key
withCredentials([sshUserPrivateKey(
    credentialsId: 'deploy-ssh-key',
    keyFileVariable: 'SSH_KEY'
)]) {
    sh 'ssh -i $SSH_KEY user@server deploy.sh'
}
```

```
// 4. Secret File (kubeconfig, certificate)
withCredentials([file(credentialsId: 'kubeconfig', variable: 'KUBECONFIG')]) {
    sh 'kubectl apply -f deployment.yaml'
}
```

For enterprise-grade secrets management, integrate Jenkins with HashiCorp Vault using the Vault plugin — secrets are fetched at runtime, never stored in Jenkins.

❗ Never use environment variables in the pipeline `environment{}` block for secrets — they appear in console logs. Always use `withCredentials{}` — it masks the values in all logs automatically.

KEY TAKEAWAYS

- ▶ Jenkins Credentials Store = encrypted secret storage, injected via `withCredentials{}`
- ▶ `withCredentials{}` auto-masks secret values in all console output
- ▶ For enterprise scale: integrate with HashiCorp Vault — secrets never touch Jenkins storage

Q14

A Jenkins pipeline is failing at the build stage — walk through your debugging approach.

● Hard

ANSWER

Systematic Jenkins build failure debugging:

- ▶ Step 1: Open Jenkins UI → click the failed build → click 'Console Output' → read the full log from the failure point upward
- ▶ Step 2: Identify the stage that failed — Jenkins Pipeline view shows red stages clearly
- ▶ Step 3: Look for the specific error message — common patterns:
 - 'sh: command not found' → tool not installed on agent (Maven, Docker, kubectl)
 - 'Permission denied' → wrong agent, missing sudo, file permission issue
 - 'Connection refused' → service not running, wrong URL, firewall
 - 'No such file or directory' → wrong working directory, missing checkout step
- ▶ Step 4: Check which agent the build ran on — label mismatch causes wrong agent
- ▶ Step 5: Reproduce locally — SSH to the agent and run the exact same command manually
- ▶ Step 6: Check environment variables — use `sh 'env'` step to print all vars
- ▶ Step 7: Check credentials — `withCredentials` block — are credential IDs correct?
- ▶ Step 8: Compare with last successful build — what changed? (git diff, config change, plugin update)

```
// Debugging helper steps in Jenkinsfile
stage('Debug') {
    steps {
        sh 'env | sort'           // print all environment variables
        sh 'docker version'      // verify tool availability
        sh 'whoami && pwd && ls -la' // verify agent context
        sh 'cat /etc/hosts'       // verify network config
    }
}
```

KEY TAKEAWAYS

- ▶ Console Output is always the first stop — read from error upward
- ▶ 'Command not found' = tool missing on agent; 'Permission denied' = wrong user/agent

- Reproduce on agent directly via SSH — eliminates Jenkins as the variable

SECTION 03

GitHub Actions

Workflows, runners, secrets, matrix builds, reusable workflows, caching

Q15**What is GitHub Actions and how is it different from Jenkins?**

● Easy

ANSWER

GitHub Actions is GitHub's native CI/CD platform. Workflows are defined in YAML files stored in `.github/workflows/` and triggered by GitHub events (push, PR, release, schedule). No separate CI server needed.

Aspect	GitHub Actions	Jenkins
Hosting	Fully managed by GitHub (or self-hosted)	Self-hosted — you manage the server
Setup	Zero — just add <code>.yml</code> file to repo	Install server, configure, maintain
Cost	Free tier + per-minute billing	Free but infrastructure cost
Integration	Native GitHub — PRs, Issues, Releases	Plugin-based GitHub integration
Marketplace	10,000+ pre-built Actions	1,800+ plugins
Scaling	GitHub manages runner scaling	You manage agent scaling
Best For	GitHub-hosted projects, open source	Enterprise, complex pipelines, on-prem
Learning Curve	Low — YAML-based	Higher — Groovy, infrastructure management

KEY TAKEAWAYS

- ▶ GitHub Actions = no server to manage; Jenkins = self-hosted, full control
- ▶ Actions uses GitHub-hosted runners for free (with limits); Jenkins needs your own agents
- ▶ For GitHub projects, Actions is the natural choice — deep GitHub integration

Q16**What are the core components of a GitHub Actions workflow — events, jobs, steps, runners?**

● Medium

ANSWER

```
// Complete GitHub Actions workflow anatomy
name: CI/CD Pipeline

# TRIGGER (event)
on:
  push:
```

```
  branches: [main, develop]
pull_request:
  branches: [main]
schedule:
  - cron: '0 2 * * *'    # nightly at 2am UTC

# ENVIRONMENT VARIABLES (global)
env:
  REGISTRY: ghcr.io
  IMAGE_NAME: myapp

jobs:
  # JOB 1
  build-and-test:
    runs-on: ubuntu-latest    # RUNNER
    steps:
      - name: Checkout code    # STEP
        uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Build with Maven
        run: mvn clean package

      - name: Run tests
        run: mvn test

      - name: Upload test results
        uses: actions/upload-artifact@v4
        with:
          name: test-results
          path: target/surefire-reports/

  # JOB 2 — depends on job 1
  deploy:
    runs-on: ubuntu-latest
    needs: build-and-test    # dependency
    if: github.ref == 'refs/heads/main'    # condition
    steps:
      - name: Deploy to staging
        run: ./scripts/deploy.sh staging
```

KEY TAKEAWAYS

- ▶ Workflow = YAML file in .github/workflows/
- ▶ Jobs run in parallel by default; use 'needs:' to create dependencies
- ▶ Steps within a job run sequentially; 'uses:' = Action from Marketplace; 'run:' = shell command

Q17**What is the difference between a GitHub-hosted runner and a self-hosted runner?**

● Medium

ANSWER

Aspect	GitHub-Hosted Runner	Self-Hosted Runner
Infrastructure	GitHub manages — fresh VM per job	You manage — your server/EC2/pod
Cost	Free minutes + billed after limit	Your infrastructure cost
Environment	ubuntu-latest, windows-latest, macos-latest	Any OS, any hardware
Pre-installed Tools	Common tools: Docker, Node, Python, Java	Whatever you install
Network Access	Public internet only	Private network, VPC, on-prem access
Security	Isolated VM, discarded after job	Persistent — must manage security
Scale	GitHub handles auto-scaling	You manage capacity
Best For	Open source, simple builds	Private network, custom hardware, GPU, cost optimization

```
// Registering a self-hosted runner
# GitHub UI: Settings → Actions → Runners → New self-hosted runner
# On your machine:
./config.sh --url https://github.com/your-org/your-repo --token TOKEN
./run.sh

# In workflow: target self-hosted runner by label
runs-on: self-hosted
# Or with custom labels:
runs-on: [self-hosted, linux, gpu]
```

KEY TAKEAWAYS

- ▶ GitHub-hosted = zero maintenance, fresh env per job, public internet only
- ▶ Self-hosted = private network access, custom tools, persistent env, your cost
- ▶ Use self-hosted when: VPC access needed, private registries, GPU workloads, cost at scale

Q18**What are GitHub Actions Secrets and how do you use them in a workflow?**

● Medium

ANSWER

GitHub Actions Secrets are encrypted environment variables stored at the repository, environment, or organization level. Secret values are never shown in logs — they are masked automatically.

Secret Scope	Where Defined	Access Level
Repository secret	Repo Settings → Secrets	All workflows in that repo
Environment secret	Repo Settings → Environments	Only workflows targeting that environment
Organization secret	Org Settings → Secrets	Selected repos in the organization
GitHub token	Auto-injected by Actions	Every workflow — GITHUB_TOKEN

```
// Using secrets in a workflow
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production # triggers environment protection rules
    steps:
      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }

      - name: Deploy to AWS
        env:
          AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
          AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
        run: aws eks update-kubeconfig --name prod-cluster

      - name: Use auto-injected GitHub token
        run: gh pr comment ${ github.event.pull_request.number } --body 'Deployed!'
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

KEY TAKEAWAYS

- Secrets are encrypted at rest and masked in all log output automatically
- GITHUB_TOKEN is auto-injected — use for GitHub API calls, PR comments, releases
- Environment secrets add an extra protection gate — only workflows targeting that environment can access them

Q19**What is a reusable workflow in GitHub Actions and when do you use it?**

● Hard

ANSWER

A reusable workflow is a GitHub Actions workflow that can be called from other workflows — similar to a function. It eliminates copy-paste pipeline duplication across multiple repositories in an organization.

```
// Reusable workflow definition (.github/workflows/deploy.yml)
name: Reusable Deploy Workflow

on:
  workflow_call:      # ← makes it reusable
  inputs:
    environment:
      required: true
      type: string
    image-tag:
      required: true
      type: string
  secrets:
    KUBE_CONFIG:
      required: true

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to ${ inputs.environment }
        env:
          KUBECONFIG_DATA: ${ secrets.KUBE_CONFIG }
        run: |
          echo $KUBECONFIG_DATA | base64 -d > kubeconfig
          kubectl set image deployment/app app=${ inputs.image-tag }
```



```
// Calling the reusable workflow
jobs:
  deploy-staging:
    uses: my-org/shared-workflows/.github/workflows/deploy.yml@main
    with:
      environment: staging
      image-tag: myapp:${ github.sha }
    secrets:
      KUBE_CONFIG: ${ secrets.STAGING_KUBE_CONFIG }
```

KEY TAKEAWAYS

- ▶ Reusable workflow = workflow_call trigger — callable from other workflows
- ▶ Accepts typed inputs and secrets — type-safe, explicit interface
- ▶ Use for: deploy logic, security scans, notifications shared across multiple repos

Q20

How do you implement matrix builds in GitHub Actions — what problem does it solve?

● Hard

ANSWER

Matrix builds run the same job with different combinations of variables simultaneously — testing across multiple OS, language versions, or configurations in parallel without duplicating workflow code.

```
// Matrix build example
jobs:
  test:
    runs-on: ${ matrix.os }
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        node-version: ['18', '20', '22']
        exclude:
          - os: windows-latest
            node-version: '22' # skip this specific combo
      fail-fast: false # don't cancel other matrix jobs if one fails
    steps:
      - uses: actions/checkout@v4
      - name: Use Node ${ matrix.node-version } on ${ matrix.os }
        uses: actions/setup-node@v4
        with:
          node-version: ${ matrix.node-version }
      - run: npm ci && npm test

# Result: 3 OS × 3 versions - 1 exclusion = 8 parallel jobs
```

Problem it solves: Instead of maintaining 9 separate jobs for 3 OS × 3 Node versions, one matrix definition runs all combinations in parallel. Coverage increases, code stays DRY.

KEY TAKEAWAYS

- ▶ Matrix = run same job with different variable combinations in parallel
- ▶ exclude: removes specific combinations; fail-fast: false lets all matrix jobs complete
- ▶ Ideal for multi-version, multi-OS, multi-region test coverage

Q21
What are GitHub Actions artifacts and caching — how do they improve pipeline performance?

● Medium

ANSWER

Feature	Artifacts	Caching
Purpose	Pass files between jobs; download after workflow	Speed up steps by reusing previously downloaded files
Persistence	Retained for configured days (default 90)	Retained per cache key, auto-evicted after 7 days no use
Use Case	Test reports, build outputs, Docker images	node_modules, Maven .m2, pip packages, Go modules
Shared Between	Jobs in same workflow run	Workflow runs on same branch
Size Limit	500MB per artifact	10GB per repo

```
// Caching and artifacts example
```

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # CACHE — restore Maven dependencies
      - name: Cache Maven packages
        uses: actions/cache@v4
        with:
          path: ~/.m2
          key: ${ runner.os }}-maven-${ hashFiles('**/pom.xml') }}
          restore-keys: ${ runner.os }}-maven-

      - run: mvn clean package

      # ARTIFACT — pass JAR to deploy job
      - name: Upload JAR
        uses: actions/upload-artifact@v4
        with:
          name: app-jar
          path: target/app.jar

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - name: Download JAR
        uses: actions/download-artifact@v4
        with:
          name: app-jar
```

KEY TAKEAWAYS

- ▶ Cache = reuse deps between runs (faster); Artifact = pass files between jobs in same run
- ▶ Cache key = hash of dependency file (pom.xml, package-lock.json) — invalidates when deps change
- ▶ Without caching, Maven/npm downloads on every run — minutes wasted per pipeline

SECTION 04

GitLab CI/CD

.gitlab-ci.yml, runners, stages, environments, Auto DevOps

Q22**What is GitLab CI/CD and how does .gitlab-ci.yml work?**

• Easy

ANSWER

GitLab CI/CD is GitLab's built-in continuous integration and delivery system. Pipelines are defined in a .gitlab-ci.yml file at the root of the repository. GitLab automatically detects this file and runs the pipeline on configured triggers.

```
// Complete .gitlab-ci.yml example
# Define stages — jobs in same stage run in parallel
stages:
  - build
  - test
  - package
  - deploy

variables:
  DOCKER_IMAGE: registry.gitlab.com/myorg/myapp

# Job in 'build' stage
build-app:
  stage: build
  image: maven:3.9-openjdk-17
  script:
    - mvn clean compile
  artifacts:
    paths:
      - target/

# Job in 'test' stage
unit-tests:
  stage: test
  image: maven:3.9-openjdk-17
  script:
    - mvn test
  artifacts:
    reports:
      junit: target/surefire-reports/*.xml

# Job in 'deploy' stage — only on main branch
deploy-production:
  stage: deploy
  script:
    - kubectl apply -f k8s/
  environment:
```

```

name: production
url: https://myapp.com
rules:
  - if: $CI_COMMIT_BRANCH == 'main'
    when: manual    # requires manual trigger

```

KEY TAKEAWAYS

- stages: defines order; jobs in same stage run in parallel
- rules: replaces only:/except: — use rules for conditional job execution
- environment: links deployment to GitLab Environments — enables deployment tracking

Q23**What are GitLab Runners and what is the difference between shared and specific runners?**● **Medium****ANSWER**

GitLab Runners are agents that execute CI/CD jobs. They are registered with GitLab and pick up jobs from the pipeline queue.

Runner Type	Scope	Managed By	Best For
Shared Runner	All projects on GitLab instance	GitLab.com (or your admin)	Small teams, open source, quick jobs
Group Runner	All projects in a GitLab group	Group maintainer	Org-wide standard pipelines
Project Runner	Specific project only	Project maintainer	Special hardware, private network, secrets isolation
Instance Runner	Entire GitLab instance	GitLab admin	Shared infrastructure for all teams

```

// Runner executors
# Shell executor — runs directly on runner machine
[[runners]]
  executor = 'shell'

# Docker executor — each job in isolated container
[[runners]]
  executor = 'docker'
  [runners.docker]
    image = 'ubuntu:22.04'

# Kubernetes executor — each job as a K8s pod
[[runners]]
  executor = 'kubernetes'
  [runners.kubernetes]
    namespace = 'gitlab-runner'

```

KEY TAKEAWAYS

- ▶ Shared runners = available to all projects; Project runners = dedicated to one project
- ▶ Docker executor = isolated, clean env per job (recommended); Shell = fast but shared state
- ▶ Tag runners with labels and use 'tags:' in jobs to route to specific runners

Q24**What are GitLab CI/CD stages, jobs, and pipelines — how do they relate?**● **Medium****ANSWER**

Pipeline → Stage → Job is the hierarchy in GitLab CI/CD.

Concept	Definition	Runs
Pipeline	Complete automation workflow triggered by an event	Once per trigger (push, MR, schedule)
Stage	Group of jobs that run together in parallel	Sequentially — next stage starts after previous completes
Job	Individual unit of work — script + config	In parallel with other jobs in same stage
Runner	Agent that executes a job	Picks up jobs from queue

```
// Stage and job relationship
stages: [build, test, deploy]

# STAGE 1: build — these jobs run in PARALLEL
build-backend:  stage: build  script: [mvn package]
build-frontend: stage: build  script: [npm run build]

# STAGE 2: test — runs AFTER all build jobs pass
unit-tests:     stage: test   script: [mvn test]
lint:           stage: test   script: [npm run lint]
security-scan:  stage: test   script: [trivy scan .]

# STAGE 3: deploy — runs AFTER all test jobs pass
deploy-staging: stage: deploy script: [kubectl apply -f k8s/staging/]
```

KEY TAKEAWAYS

- ▶ Jobs in the same stage run in parallel; stages run sequentially
- ▶ A stage fails if any job in it fails — subsequent stages are skipped
- ▶ needs: keyword creates job-level dependencies regardless of stage order

Q25**What are GitLab Environments and Deployments — how do you implement environment promotion?**● **Hard**

ANSWER

GitLab Environments track where your code is deployed. Each deployment is recorded with a status, timestamp, and link to the running app. Environment promotion is the process of moving a validated version from staging to production.

```
// Environment promotion pipeline
stages: [build, test, deploy-staging, promote-production]

build:
  stage: build
  script: docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .

deploy-staging:
  stage: deploy-staging
  script:
    - helm upgrade myapp ./chart --set image.tag=$CI_COMMIT_SHA
      --namespace staging
  environment:
    name: staging
    url: https://staging.myapp.com
  rules:
    - if: $CI_COMMIT_BRANCH == 'main'

# Production: same image as staging (promoted)
deploy-production:
  stage: promote-production
  script:
    - helm upgrade myapp ./chart --set image.tag=$CI_COMMIT_SHA
      --namespace production
  environment:
    name: production
    url: https://myapp.com
  rules:
    - if: $CI_COMMIT_BRANCH == 'main'
      when: manual # ← human approval gate
```

GitLab Environments UI shows: active deployments per environment, deployment history, rollback button, and who approved each deployment.

KEY TAKEAWAYS

- Environments = named deployment targets tracked in GitLab UI (staging, production)
- when: manual = human approval gate before production promotion
- Same image deployed to staging and promoted to production — no rebuild

Q26**What is GitLab Auto DevOps and what does it auto-configure?**

● Medium

ANSWER

GitLab Auto DevOps is a collection of pre-built CI/CD templates that automatically configure a complete pipeline for your application without writing a .gitlab-ci.yml file. GitLab detects your language and applies sensible defaults.

Auto DevOps auto-configures:

- Auto Build — Dockerfile detection or Heroku buildpacks

- Auto Test — language-specific test runner detection
- Auto Code Quality — code quality report on every MR
- Auto SAST — static security scanning
- Auto Dependency Scanning — vulnerable library detection
- Auto Container Scanning — Docker image CVE scan
- Auto DAST — dynamic security testing against deployed app
- Auto Deploy — Helm chart deployment to Kubernetes
- Auto Monitoring — Prometheus metrics setup

Scenario	Recommendation
New greenfield project	Start with Auto DevOps — override what you need
Existing pipeline	Keep custom .gitlab-ci.yml — Auto DevOps adds complexity
No Kubernetes	Disable auto-deploy; keep build/test/scan stages
Compliance-heavy	Custom pipeline — Auto DevOps may not meet audit requirements

KEY TAKEAWAYS

- Auto DevOps = zero-config complete pipeline for standard web apps
- Includes SAST, dependency scan, container scan automatically — security by default
- Can be customized by overriding individual Auto DevOps templates

SECTION 05

Pipeline Design & Deployment Strategies

Blue-green, canary, rolling, feature flags, approval gates

Q27

What is a multi-stage pipeline and why is it better than a single-stage build?

● Medium

ANSWER

A multi-stage pipeline breaks the delivery process into discrete, ordered stages where each stage must pass before the next begins. Failures are caught at the earliest, cheapest point.

Aspect	Single-Stage Build	Multi-Stage Pipeline
Failure Detection	All steps run before failure reported	Fails fast at first failing stage
Parallel Execution	Sequential	Jobs within same stage run in parallel
Deployment Safety	No gates — code goes straight to prod	Staging → validation → manual gate → prod
Feedback Speed	Full pipeline completes before feedback	Stage-level feedback in minutes
Rollback	Full re-run needed	Re-deploy last good artifact
Auditability	Limited	Each stage has its own logs and status

Typical multi-stage structure: fast feedback stages (build, lint, unit test) → slow validation stages (integration test, security scan) → deployment stages (staging, smoke test, production).

KEY TAKEAWAYS

- ▶ Multi-stage = fail fast at the cheapest point — unit tests before integration tests
- ▶ Parallel jobs in the same stage reduce total pipeline time
- ▶ Each stage is a quality gate — production only sees code that passed all previous stages

Q28

What is a blue-green deployment strategy and how do you implement it in a pipeline?

● Hard

ANSWER

Blue-green deployment runs two identical production environments — Blue (current live) and Green (new version). Traffic is switched from Blue to Green atomically after Green is validated. Rollback is instant — switch traffic back to Blue.

```
// Blue-green with Kubernetes and a pipeline
# Pipeline stage: deploy to green
deploy-green:
  script:
    - kubectl set image deployment/myapp-green
      app=myapp:$NEW_VERSION
```

```

- kubectl rollout status deployment/myapp-green

# Pipeline stage: smoke test green before switching
traffic
smoke-test-green:
  script:
    - curl -f https://green.internal.myapp.com/health
    - run-smoke-tests.sh
https://green.internal.myapp.com

# Pipeline stage: switch traffic (manual approval)
switch-traffic:
  when: manual
  script:
    # Update load balancer / ingress to point to green
    - kubectl patch service myapp-service
      -p '{"spec":{"selector":{"version":"green"}}}'

# Rollback: switch back to blue (instant)
rollback:
  when: manual
  script:
    - kubectl patch service myapp-service
      -p '{"spec":{"selector":{"version":"blue"}}}'

```

Aspect	Blue-Green	Rolling
Downtime	Zero — instant switch	Zero — gradual replacement
Resource Cost	2x infrastructure during switch	Normal — gradual replacement
Rollback Speed	Instant — flip traffic back	Slow — must roll forward all pods
Risk	Low — full validation before switch	Medium — new version serves traffic during rollout

📌 Blue-green guarantees zero downtime and instant rollback but costs 2x infrastructure during the switch window. Ideal for scheduled releases with fixed maintenance windows.

KEY TAKEAWAYS

- ▶ Blue-green = two full environments; traffic switched atomically between them
- ▶ Rollback = flip the load balancer back — seconds, not minutes
- ▶ Cost: 2x infra during deployment; Risk: lowest of all strategies

Q29

What is a canary deployment and how is it different from blue-green?

● **Hard**

ANSWER

Canary deployment routes a small percentage of traffic (e.g., 5%) to the new version while the rest continues on the old version. The percentage is gradually increased as confidence grows. If metrics degrade, rollback immediately.

```
// Canary with Kubernetes and Argo Rollouts
# ArgoCD Rollout definition
apiVersion: argoproj.io/v1alpha1
kind: Rollout
spec:
  strategy:
    canary:
      steps:
        - setWeight: 5      # 5% traffic to new
          version
        - pause: {duration: 10m} # wait 10 minutes
        - setWeight: 25      # 25% if no issues
        - pause: {duration: 10m}
        - setWeight: 50
        - pause: {duration: 10m}
        - setWeight: 100    # full rollout
      analysis:             # auto-rollback on metric
        failure
      templates:
        - templateName: error-rate
      args:
        - name: service-name
          value: myapp
```

Aspect	Canary	Blue-Green
Traffic Split	Gradual (5% → 25% → 50% → 100%)	Instant (0% → 100%)
Rollback	Instant — redirect traffic back	Instant — flip LB back
Risk Exposure	Low — only % of users see new version	Zero until switch, then all users
Infra Cost	Low — only extra pods for canary %	High — 2x full environment
Observation	Real user traffic on new version	No real traffic until switch

KEY TAKEAWAYS

- ▶ Canary = gradual traffic shift with automatic rollback on metric degradation
- ▶ Real production traffic validates the new version before full rollout
- ▶ Canary costs less than blue-green but exposes a % of users to the new version

Q30

What is a rolling deployment and when does it cause issues?

● Medium

ANSWER

A rolling deployment gradually replaces old version pods with new version pods, one batch at a time. At any given moment during the rollout, both old and new versions are running and serving traffic.

```
// Kubernetes rolling deployment config
spec:
  replicas: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2      # max pods above desired count during update
      maxUnavailable: 1 # max pods that can be down during update
  template:
    spec:
      containers:
      - name: app
        image: myapp:v2 # new version
```

When rolling deployments cause issues:

- ▶ Database migrations — if v2 changes the DB schema, v1 pods reading new schema fail
 - ▶ API breaking changes — v1 and v2 handling requests simultaneously with incompatible behavior
 - ▶ Session state — user sessions created on v1 may fail on v2 if session format changed
 - ▶ Long rollback — can't instantly switch back; must roll forward or wait for full reversal
- Mitigation: always make migrations backward-compatible, use expand-contract pattern for DB changes.

KEY TAKEAWAYS

- ▶ Rolling = gradual pod replacement — zero downtime but both versions run simultaneously
- ▶ Risk: backward-incompatible DB migrations or API changes cause errors during rollout
- ▶ Use expand-contract pattern: add new DB columns before deploying new code, remove old ones after

Q31

What are pipeline gates and approval steps — when and how do you use them?

● Medium

ANSWER

Pipeline gates are checkpoints that pause the pipeline and require a condition to be met before proceeding. They enforce quality standards and compliance requirements at defined points.

Gate Type	Trigger	Example
Manual approval	Human clicks 'Approve' in UI	Production deployment — manager approval
Quality gate	SonarQube score threshold	Code coverage > 80%, no critical issues
Test gate	All tests must pass	Integration tests must be green
Security gate	No high/critical CVEs	Container scan must pass before deploy
Time gate	Deployment window constraint	Only deploy Mon-Thu 10am-4pm

Notification gate

Slack/email
confirmationNotify team, wait for no-
objection

```
// Manual approval gate in GitHub Actions
deploy-production:
  runs-on: ubuntu-latest
  environment:
    name: production    # environment has required
    reviewers configured
  needs: [test, security-scan, deploy-staging]
  steps:
    - name: Deploy to production
      run: helm upgrade myapp ./chart --namespace
production
```

In GitHub Actions, environment protection rules (required reviewers) act as manual approval gates automatically.

KEY TAKEAWAYS

- ▶ Gates enforce quality and compliance before code reaches the next stage
- ▶ Manual approval gates are essential for production in Continuous Delivery
- ▶ Environment protection rules in GitHub Actions = automatic approval gate with named reviewers

SECTION 06

Docker & Kubernetes in CI/CD

Docker build in pipelines, image scanning, Helm, kubectrl, rollbacks

Q32**How do you build and push a Docker image in a CI/CD pipeline?**

● Medium

ANSWER

Docker image building in CI/CD follows a consistent pattern: authenticate → build → tag → scan → push.

```
// GitHub Actions Docker build and push
name: Build and Push Docker Image

jobs:
  docker:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # Login to registry
      - name: Login to Amazon ECR
        uses: aws-actions/amazon-ecr-login@v2

      # Build with cache
      - name: Build Docker image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: false
          tags: |
            ${ env.ECR_REGISTRY }/myapp:latest
            ${ env.ECR_REGISTRY }/myapp:${ github.sha }
          cache-from: type=gha
          cache-to: type=gha,mode=max

      # Scan before push
      - name: Scan image with Trivy
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: ${ env.ECR_REGISTRY }/myapp:${ github.sha }
          exit-code: '1'
          severity: 'HIGH,CRITICAL'

      # Push only after scan passes
      - name: Push to ECR
        run: docker push ${ env.ECR_REGISTRY }/myapp:${ github.sha }
```

📌 Tag images with the Git SHA (\${{ github.sha }}) — not 'latest'. Latest is mutable and untrackable. SHA tags are immutable — you always know exactly which commit is in production.

KEY TAKEAWAYS

- ▶ Always scan before push — catch vulnerabilities before they reach the registry
- ▶ Tag with Git SHA — immutable, traceable; never rely on 'latest' in production
- ▶ Docker layer caching in CI (cache-from/cache-to gha) can cut build time by 60-80%

Q33**What is a multi-stage Dockerfile and why is it important for CI/CD?**

● Medium

ANSWER

A multi-stage Dockerfile uses multiple FROM instructions to create intermediate build stages, with only the final stage's filesystem included in the produced image. This dramatically reduces image size and attack surface.

```
// Multi-stage Dockerfile
# Stage 1: Build
FROM maven:3.9-openjdk-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:resolve # cache layer for deps
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Runtime (minimal image)
FROM eclipse-temurin:17-jre-alpine AS runtime
WORKDIR /app

# Copy ONLY the JAR from builder — not Maven, JDK, source code
COPY --from=builder /app/target/app.jar ./app.jar

# Non-root user for security
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

# Result: builder ~600MB → runtime image ~150MB
```

Benefits for CI/CD: smaller images push faster to registries, pull faster to Kubernetes nodes, have smaller CVE surface, and reduce network transfer costs.

KEY TAKEAWAYS

- ▶ Multi-stage = build in one stage, run in minimal image — massive size reduction
- ▶ Build tools (Maven, JDK full, node_modules) never enter the production image
- ▶ Always use non-root USER in the runtime stage — required for most security policies

Q34**What is Helm and how does it integrate with CI/CD pipelines?**

● Hard

ANSWER

Helm is the package manager for Kubernetes. It templates Kubernetes YAML manifests using Go templates, packages them as 'charts', and manages releases with versioning, upgrade, and rollback capabilities.

```
// Helm in CI/CD pipeline
# Pipeline deploy stage
deploy-staging:
  script:
    # Deploy or upgrade (idempotent)
    - helm upgrade --install myapp ./helm/myapp
      --namespace staging
      --create-namespace
      --set image.tag=$CI_COMMIT_SHA
      --set replicaCount=2
      --values helm/values-staging.yaml
      --wait                # wait for pods ready
      --timeout 5m
      --atomic              # auto-rollback if deploy
  fails

# Values file per environment
# helm/values-staging.yaml
image:
  repository: registry.io/myapp
  tag: latest    # overridden by --set image.tag=$SHA
replicaCount: 2
resources:
  limits: {cpu: 500m, memory: 512Mi}
ingress:
  host: staging.myapp.com
```

Helm Feature	Benefit in CI/CD
--atomic flag	Auto-rollback if deployment fails health checks
--wait flag	Pipeline waits for pods to be Ready before success
values per env	Same chart, different values.yaml for staging/prod
helm rollback	One command to revert to previous release
helm history	Full deployment history with timestamps and revisions

KEY TAKEAWAYS

- ▶ helm upgrade --install = deploy or upgrade idempotently
- ▶ --atomic = auto-rollback if Kubernetes reports unhealthy pods
- ▶ One chart + environment-specific values.yaml = DRY Kubernetes deployments

Q35**How do you implement a Kubernetes rollback in a CI/CD pipeline?**

● Hard

ANSWER

Kubernetes and Helm both provide rollback mechanisms. A CI/CD pipeline should automate rollback detection and execution.

```
// Rollback approaches
# 1. Helm rollback - to previous release
helm rollback myapp 0 # 0 = previous revision
helm rollback myapp 5 # specific revision

# 2. Kubernetes native rollback
kubectl rollout undo deployment/myapp
kubectl rollout undo deployment/myapp --to-revision=3

# 3. Pipeline auto-rollback (--atomic in Helm)
helm upgrade --install myapp ./chart --atomic --timeout 5m
# If pods fail readiness: Helm auto-rolls back and pipeline fails

# 4. Pipeline with explicit rollback job
deploy:
  script:
    - helm upgrade --install myapp ./chart --wait || (helm rollback myapp 0 && exit 1)

# 5. ArgoCD auto-rollback
# Configure in ArgoCD Application spec:
spec:
  syncPolicy:
    automated:
      selfHeal: true # reverts manual changes
```

Best practice: Use --atomic in Helm deploys. This makes the pipeline stage fail automatically if the deployment is unhealthy, triggering whatever on-failure logic (alerts, rollback) you've defined.

KEY TAKEAWAYS

- ▶ helm rollback myapp 0 = revert to previous Helm release revision
- ▶ kubectl rollout undo = Kubernetes native rollback without Helm
- ▶ --atomic flag in Helm = automatic rollback + pipeline failure if deploy unhealthy

Q36**What is Docker-in-Docker (DinD) and what are its security risks in CI/CD?**

● Hard

ANSWER

Docker-in-Docker (DinD) runs a Docker daemon inside a Docker container, allowing CI/CD jobs running in containers to build Docker images. It is commonly needed when Jenkins or GitLab runners themselves run in Docker containers.

Approach	How	Security	Performance
Docker-in-Docker (DinD)	Run privileged Docker daemon inside CI container	HIGH risk — privileged container has host access	Slow — new daemon per job

Docker socket mount	Mount /var/run/docker.sock from host into CI container	MEDIUM risk — can access all host containers	Fast — uses host daemon
Kaniko	Build images without Docker daemon — runs as unprivileged	LOW risk — no daemon needed	Moderate — pure container
BuildKit / Buildah	Daemonless Docker-compatible build	LOW risk — rootless builds	Fast

```
// Kaniko in Kubernetes pipeline (recommended)
# GitLab CI with Kaniko
build-image:
  image:
    name: gcr.io/kaniko-project/executor:v1.20.0-debug
    entrypoint: ['']
  script:
    - /kaniko/executor
      --context $CI_PROJECT_DIR
      --dockerfile Dockerfile
      --destination
    $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
```

📌 For Kubernetes-based CI/CD, use Kaniko or Buildah instead of DinD. DinD requires privileged mode which violates Kubernetes security policies (PodSecurityPolicy/Pod Security Admission) in most production clusters.

KEY TAKEAWAYS

- ▶ DinD = Docker daemon in container — requires privileged mode, security risk
- ▶ Socket mount = shares host Docker — faster but container has host-level access
- ▶ Kaniko = recommended for K8s CI — builds images without daemon, no privileged mode needed

Q37
How do you manage environment-specific configuration in Kubernetes deployments from a pipeline?

● Medium

ANSWER

Environment-specific configuration in Kubernetes should be injected at deploy time, never baked into the image. Common patterns:

Pattern	Tool	How	Best For
Values files	Helm	--values staging.yaml, --values prod.yaml	Kubernetes-native apps with Helm
ConfigMaps	kubectl/Helm	Mounted as files or env vars	Non-sensitive config

Secrets	Kubernetes Secrets / Vault	Mounted at runtime, never in image	Sensitive values
Kustomize	kustomize	Overlays per environment on base manifests	Non-Helm environments
External Secrets	External Secrets Operator	Sync secrets from Vault/AWS SM	Enterprise secrets management

```
// Pipeline injecting config at deploy
deploy-staging:
  script:
    - helm upgrade --install myapp ./chart
      --values helm/values-base.yaml      # shared
    config
      --values helm/values-staging.yaml    #
    staging overrides
      --set image.tag=$COMMIT_SHA          #
    pipeline var
      --set database.host=$STAGING_DB_HOST # from
    CI secret
      --namespace staging
```

KEY TAKEAWAYS

- ▶ Never bake environment config into the image — inject at deploy time
- ▶ Helm values files = cleanest pattern; base.yaml + environment overlay
- ▶ Sensitive values → Kubernetes Secrets or External Secrets Operator — never in values.yaml in Git

SECTION 07

Testing in CI/CD

Test types, parallelization, SonarQube, shift-left, test reporting

Q38**What types of tests should run at each stage of a CI/CD pipeline?**

● Medium

ANSWER

Stage	Test Type	Speed	Dependencies	Goal
Commit stage	Unit tests + linting	Fast (<5 min)	None — mocked	Fast developer feedback
Build stage	Static analysis, SAST	Medium (5-10 min)	None	Code quality gate
Integration stage	Integration tests	Slow (10-30 min)	Real DB, services	Business logic validation
Staging	E2E tests, smoke tests	Slow (20-60 min)	Full environment	User journey validation
Pre-production	Performance tests	Very slow (hours)	Production-like env	Capacity validation
Production	Synthetic monitoring	Continuous	Production	Always-on health check

Test pyramid principle: many fast unit tests → fewer integration tests → even fewer E2E tests. Invert the pyramid and your pipeline is slow and flaky.

🔗 'Fail fast' = run fastest tests first. Unit tests in 2 minutes tell the developer immediately. Don't run 45-minute E2E tests before unit tests — that wastes everyone's time.

KEY TAKEAWAYS

- ▶ Unit tests first — fastest feedback; E2E tests last — slowest but most realistic
- ▶ Test pyramid: many unit → some integration → few E2E
- ▶ Synthetic monitoring in production = continuous validation, not just at deploy time

Q39**What is test parallelization in CI/CD and how does it reduce pipeline time?**

● Hard

ANSWER

Test parallelization splits a test suite across multiple agents/containers that run simultaneously, reducing total test execution time proportionally to the number of parallel workers.

```
// GitHub Actions parallel test execution
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        shard: [1, 2, 3, 4] # 4 parallel shards
    steps:
      - uses: actions/checkout@v4
      - name: Run test shard ${ matrix.shard } of 4
        run: |
          # Run 1/4 of total tests per shard
          pytest tests/ --splits 4 --group ${ matrix.shard }

# After all shards complete
merge-results:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - name: Merge coverage reports
      run: coverage combine && coverage report
```

Scenario	Sequential	4 Parallel Shards	8 Parallel Shards
1000 tests	40 minutes	10 minutes	5 minutes
Cost	1 runner	4 runners (4x cost)	8 runners (8x cost)
Sweet Spot	<100 tests	100-2000 tests	2000+ tests

KEY TAKEAWAYS

- ▶ Parallelization = same tests split across N workers running simultaneously
- ▶ N shards reduces time by ~N× but costs N× the runner minutes
- ▶ Use pytest-split, Jest --shard, or JUnit parallel runner for test sharding

Q40**What is a quality gate in CI/CD — how does SonarQube integrate into a pipeline?**

● Hard

ANSWER

A quality gate is a set of conditions that code must meet to pass. SonarQube is the most common tool that implements quality gates in CI/CD pipelines — analyzing code for bugs, vulnerabilities, code smells, and coverage.

```
// SonarQube integration in GitHub Actions
- name: SonarQube Scan
  uses: SonarSource/sonarqube-scan-action@master
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
    SONAR_HOST_URL: ${ secrets.SONAR_HOST_URL }
  with:
```

```

args: >
  -Dsonar.projectKey=myapp
  -Dsonar.sources=src
  -Dsonar.tests=test
  -
Dsonar.coverage.jacoco.xmlReportPaths=target/jacoco.xml

- name: Quality Gate Check
  uses: SonarSource/sonarqube-quality-gate-action@master
  timeout-minutes: 5
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
  # Fails the pipeline if quality gate fails

```

SonarQube Metric	Typical Gate Threshold
Coverage	New code $\geq$ 80% line coverage
Duplications	New code $\leq$ 3% duplication
Maintainability	Rating $\geq$ A (technical debt ratio < 5%)
Reliability	Rating $\geq$ A (zero new bugs)
Security	Rating $\geq$ A (zero new vulnerabilities)

KEY TAKEAWAYS

- Quality gate = automated pass/fail decision based on code metrics
- SonarQube scans → posts to PR → fails pipeline if gate conditions not met
- Always set gates on NEW code — don't fail pipeline for pre-existing legacy issues

Q41**What is shift-left testing and what does it mean for pipeline design?**

● Medium

ANSWER

Shift-left testing means moving testing activities earlier in the development lifecycle — towards the left of the timeline. Instead of testing at the end (before release), developers test continuously from the first commit.

Traditional (Shift-Right)	Shift-Left
Developers write code for weeks	Developers write tests alongside code
QA tests after development complete	Unit tests run on every commit
Bugs found late — expensive to fix	Bugs found in minutes — cheap to fix
Security audit before release	SAST/SCA scanning on every PR
Performance test before go-live	Load tests in staging pipeline

Shift-left in CI/CD pipeline design means:

- Unit tests in the first stage — before anything else
- SAST (static security scan) in the build stage — not in a separate security phase

- ▶ Dependency vulnerability scanning on every PR — not quarterly audits
- ▶ Code quality gates before code reaches staging — not after
- ▶ Contract tests for microservices — validate API compatibility before integration

KEY TAKEAWAYS

- ▶ Shift-left = move testing earlier — faster feedback, cheaper bug fixes
- ▶ A bug found in commit stage costs minutes; same bug in production costs days
- ▶ SAST, coverage, quality gates in commit/build stages — not as separate late-stage checks

Q42**How do you handle flaky tests in a CI/CD pipeline?**

● Medium

ANSWER

Flaky tests are tests that pass sometimes and fail other times without code changes. They erode trust in the pipeline, cause unnecessary retries, and slow down delivery.

Strategies to handle flaky tests:

- ▶ Identify flaky tests — track test failure rate over multiple runs; flag tests that fail >5% without code change
- ▶ Quarantine — move flaky tests to a separate job that doesn't block the pipeline. Fix them separately
- ▶ Retry logic — retry flaky tests automatically (1-2 retries max) while tracking the retry rate
- ▶ Fix root cause — most flakiness is caused by: timing issues (add explicit waits, not sleep()), shared state (isolate test data), external dependencies (mock them), port conflicts (use dynamic ports)

```
// Retry in GitHub Actions
- name: Run E2E tests (with retry)
  uses: nick-fields/retry@v3
  with:
    timeout_minutes: 10
    max_attempts: 3
    command: npm run test:e2e

# pytest retry
pytest tests/ --reruns 2 --reruns-delay 5

# JUnit retry with Maven Surefire
mvn test -Dsurefire.rerunFailingTestsCount=2
```

KEY TAKEAWAYS

- ▶ Flaky tests = intermittently failing tests unrelated to code changes
- ▶ Quarantine flaky tests immediately — don't let them block the main pipeline
- ▶ Fix root cause: replace sleep() with explicit waits, isolate test data, mock external services

SECTION 08

Security & Secrets Management

DevSecOps, SAST, DAST, container scanning, Vault, supply chain security

Q43

What is DevSecOps and how do you integrate security scanning into a CI/CD pipeline?

● Hard

ANSWER

DevSecOps integrates security practices into every stage of the CI/CD pipeline — security becomes a shared responsibility between Dev, Sec, and Ops rather than a final checkpoint.

Security Layer	Tool	Pipeline Stage	What It Catches
SAST	SonarQube, Semgrep, CodeQL	Build/commit	SQL injection, XSS, insecure code patterns
SCA	OWASP Dependency Check, Snyk	Build	Vulnerable open-source libraries
Container Scan	Trivy, Gype, Snyk	After Docker build	CVEs in base image and packages
DAST	OWASP ZAP, Burp Suite	Staging	Running app vulnerabilities
Secret Detection	TruffleHog, detect-secrets, Gitleaks	Commit/PR	Hardcoded passwords, API keys in code
IaC Scan	Checkov, tfsec, kics	Before Terraform apply	Misconfigured infra (open S3, no encryption)

📌 The key DevSecOps answer: security shifts left. SAST runs on every commit, not in a quarterly audit. Container scanning happens before push, not in production. Secret detection runs on every PR, not after a breach.

KEY TAKEAWAYS

- ▶ DevSecOps = security at every pipeline stage, not just at the end
- ▶ SAST = code analysis (no running app); DAST = live app testing (needs running app)
- ▶ Container scanning before push prevents vulnerable images from reaching the registry

Q44

What is SAST vs DAST — where does each fit in a CI/CD pipeline?

● Hard

ANSWER

Aspect	SAST (Static Application Security Testing)	DAST (Dynamic Application Security Testing)
What it tests	Source code — without running the app	Running application — live HTTP testing

When in pipeline	Early — commit or build stage	Late — staging environment
Requires	Source code access	Deployed, running application
Finds	Insecure code patterns, SQL injection in code, hardcoded secrets	Injection attacks, auth bypasses, session issues in running app
False Positives	Higher — static analysis guesses runtime	Lower — actual runtime behavior tested
Speed	Fast (minutes)	Slow (30+ minutes for full scan)
Tools	SonarQube, Semgrep, CodeQL, Checkmarx	OWASP ZAP, Burp Suite, Nikto

```
// SAST in pipeline (GitHub Actions)
- name: SAST with Semgrep
  uses: returntocorp/semgrep-action@v1
  with:
    config: p/owasp-top-ten

# DAST with OWASP ZAP against staging
- name: ZAP DAST Scan
  uses: zaproxy/action-full-scan@v0.10.0
  with:
    target: 'https://staging.myapp.com'
    rules_file_name: '.zap/rules.tsv'
    cmd_options: '-a'
```

KEY TAKEAWAYS

- ▶ SAST = static code analysis (no running app) → early in pipeline
- ▶ DAST = live app attack simulation → staging environment
- ▶ Use both: SAST catches what's in code; DAST catches what's in behavior

Q45**What is secrets management in CI/CD — how does HashiCorp Vault integrate with pipelines?**

● Hard

ANSWER

Secrets management is the practice of securely storing, accessing, and rotating sensitive values (passwords, API keys, certificates) used by pipelines and applications. HashiCorp Vault is the enterprise standard for centralized secrets management.

```
// Vault integration in GitLab CI
# .gitlab-ci.yml with Vault integration
deploy:
  image: vault:latest
  variables:
    VAULT_ADDR: 'https://vault.company.com'
  id_tokens:
```

```

VAULT_ID_TOKEN:
  aud: https://vault.company.com
secrets:
  # Fetch from Vault at runtime
  DATABASE_PASSWORD:
    vault: myapp/staging/database/password@secret
  AWS_SECRET_KEY:
    vault: aws/staging/secret-key@secret
script:
  - echo 'Deploying with secrets from Vault'
  - deploy.sh

```

Approach	Security Level	Rotation	Audit Trail
Hardcoded in code	None — NEVER do this	Manual	None
CI/CD built-in secrets	Medium — encrypted, not rotated	Manual	Limited
HashiCorp Vault	High — encrypted, dynamic, rotated	Automatic	Full audit log
AWS Secrets Manager	High — cloud-native, IAM-controlled	Automatic	CloudTrail

KEY TAKEAWAYS

- ▶ Vault = centralized secrets with dynamic generation, auto-rotation, full audit log
- ▶ Dynamic secrets: Vault generates a DB password per pipeline run — it expires after use
- ▶ Never store secrets in Git, Dockerfile, or CI/CD YAML files — ever

Q46**What is supply chain security in CI/CD — what is SLSA and how do you implement it?**

● Hard

ANSWER

Supply chain security protects the CI/CD pipeline itself from attack — ensuring that the code, dependencies, and build process have not been tampered with. SLSA (Supply-chain Levels for Software Artifacts) is a framework defining security levels.

SLSA Level	Requirements	What It Prevents
Level 1	Provenance generated — build process documented	Accidental mistakes
Level 2	Hosted build service (GitHub Actions, GitLab CI) with signed provenance	Tampering after build
Level 3	Hermetic build — no network access during build; signed artifacts	Compromised build environment

Level 4

Two-party review; hermetic reproducible builds

Insider threats, compromised CI

Key supply chain security practices:

- ▶ Pin Action versions to SHA (not tags): uses: actions/checkout@a81bbbf — not @v4 (tag is mutable)
- ▶ Sign container images with Sigstore/Cosign
- ▶ Generate SBOM (Software Bill of Materials) — inventory of all dependencies
- ▶ Dependency pinning — lock file (package-lock.json, go.sum) committed to Git
- ▶ Artifact signing — sign JARs, Docker images with private key

KEY TAKEAWAYS

- ▶ Supply chain attacks target the build pipeline, not the app code (SolarWinds, Log4Shell)
- ▶ Pin GitHub Action versions to SHA — tags are mutable and can be hijacked
- ▶ SBOM = inventory of every dependency — required for enterprise security compliance

SECTION 09

GitOps & ArgoCD

GitOps principles, ArgoCD, Flux, sync policies, real-world scenarios

Q47

What is GitOps and how does it differ from traditional CI/CD?

● Medium

ANSWER

GitOps is a deployment methodology where Git is the single source of truth for both application code AND infrastructure/deployment configuration. A GitOps operator (ArgoCD, Flux) continuously syncs the cluster state to match what's in Git.

Aspect	Traditional CI/CD Push	GitOps Pull
Deployment trigger	CI pipeline pushes to cluster	GitOps operator pulls from Git
Source of truth	CI pipeline + cluster state	Git repository — always
Drift detection	Manual or periodic checks	Continuous — alerts on drift immediately
Rollback	Re-run old pipeline	git revert — cluster follows automatically
Audit trail	CI logs	Full Git history — who changed what, when, why
Access to cluster	CI server has cluster credentials	Only GitOps operator has cluster access
Cluster recovery	Rebuild from pipeline state	Re-point operator to Git — cluster self-heals

🔑 Key GitOps interview point: in GitOps, the CI pipeline does NOT push to Kubernetes. It updates the Git repo (e.g., image tag in values.yaml). The GitOps operator sees the change and syncs the cluster. CI = build + test; GitOps = deploy.

KEY TAKEAWAYS

- ▶ GitOps = Git is the only source of truth; GitOps operator deploys continuously
- ▶ Traditional CI/CD pushes to cluster; GitOps operator PULLS from Git
- ▶ Drift detection is automatic — any manual kubectl change is detected and reverted

Q48

What is ArgoCD and how does it implement GitOps?

● Medium

ANSWER

ArgoCD is a declarative GitOps operator for Kubernetes. It runs inside the cluster, watches a Git repository for changes to Kubernetes manifests or Helm charts, and automatically syncs the cluster to match the desired state in Git.

```
// ArgoCD Application definition
```

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: myapp
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/myorg/k8s-configs
    targetRevision: main
    path: apps/myapp/staging
    helm:
      valueFiles:
        - values-staging.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: staging
  syncPolicy:
    automated:
      prune: true      # delete resources removed from Git
      selfHeal: true   # revert manual cluster changes
    syncOptions:
      - CreateNamespace=true
```

ArgoCD sync states: Synced (cluster matches Git), OutOfSync (drift detected), Progressing (sync in progress), Degraded (sync failed).

KEY TAKEAWAYS

- ▶ ArgoCD runs IN the cluster — it pulls from Git, never exposes cluster to external CI
- ▶ prune: true = delete resources removed from Git; selfHeal: true = revert manual changes
- ▶ ArgoCD UI shows sync status, deployment history, resource health at a glance

Q49

What is the image updater pattern in GitOps — how do you update image tags?

● Hard

ANSWER

In GitOps, the CI pipeline does not kubectl apply directly. Instead, it updates the image tag in the Git repository, and the GitOps operator picks up the change and deploys it.

```
// GitOps image update flow
# Step 1: CI pipeline builds and pushes image
- name: Build and push
  run: |
    docker build -t registry.io/myapp:${{ github.sha }} .
    docker push registry.io/myapp:${{ github.sha }}

# Step 2: CI updates the image tag in Git (k8s config repo)
- name: Update image tag in GitOps repo
  run: |
    git clone https://github.com/myorg/k8s-configs
    cd k8s-configs
    # Update the tag in values.yaml
```

```
sed -i 's/tag: .*/tag: ${github.sha}/' apps/myapp/values.yaml
git commit -am 'ci: update myapp image to ${github.sha}'
git push
```

```
# Step 3: ArgoCD detects the Git change
# → Syncs cluster with new image tag automatically
# → No CI credentials needed for cluster access
```

Alternative: ArgoCD Image Updater — a component that automatically detects new images in the registry and commits the updated tag to Git, removing the need for the CI pipeline to touch the GitOps repo.

KEY TAKEAWAYS

- ▶ CI updates Git, not the cluster — Git is the deployment trigger, not kubectl
- ▶ ArgoCD Image Updater automates image tag updates in Git — fully automated GitOps
- ▶ Separate app repo (source code) from config repo (k8s manifests) — cleaner GitOps boundary

Q50
What is Flux CD and how does it compare to ArgoCD?

● Medium

ANSWER

Aspect	ArgoCD	Flux CD
UI	Rich web UI + CLI	CLI-focused, minimal UI (Weave GitOps for UI)
Architecture	Centralized — one ArgoCD for all clusters	Distributed — Flux runs per cluster
Multi-tenancy	Projects and RBAC built-in	Tenant isolation via namespace separation
Image automation	ArgoCD Image Updater (separate component)	Flux Image Automation built-in
Notification	Notifications controller (separate)	Built-in alerts
Helm Support	Native Helm application type	HelmRelease CRD
GitOps toolkit	Monolithic application	Modular toolkit (source, kustomize, helm, notification controllers)
Best For	Teams wanting rich UI and centralized control	Teams wanting GitOps primitives and flexibility

KEY TAKEAWAYS

- ▶ Both ArgoCD and Flux implement GitOps pull-based deployment — different architecture
- ▶ ArgoCD = centralized, rich UI, great multi-cluster UX
- ▶ Flux = modular toolkit, more composable, better for platform teams building GitOps platforms

Q51
How do you handle database migrations in a CI/CD pipeline?

● Hard

ANSWER

Database migrations are one of the hardest problems in CI/CD because code and schema must stay compatible during rolling deployments when both old and new code versions serve traffic simultaneously.

The Expand-Contract pattern (recommended):

- Expand phase: Add new column/table as optional (nullable or with default). Deploy this migration BEFORE new code. Both old and new code work with the expanded schema.
- Contract phase: After all old code is gone and new code handles the new schema, remove old columns/tables.

```
// Migration in Kubernetes with Init Container
spec:
  initContainers:
  - name: db-migrate
    image: myapp:${ NEW_VERSION }
    command: ['python', 'manage.py', 'migrate']
    env:
    - name: DATABASE_URL
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: url
  containers:
  - name: app
    image: myapp:${ NEW_VERSION }
    # App starts only AFTER initContainer (migration) succeeds
```

Tools: Flyway, Liquibase, Alembic, Django migrations — all support versioned, tracked migration files committed to Git.

KEY TAKEAWAYS

- Expand-contract = add new columns first (backward compatible), remove old ones later
- Init containers run migrations before the app starts — atomic with deployment
- Never add NOT NULL columns without defaults to a running production table — breaks rolling deploy

Q52

What is a CI/CD pipeline for a microservices architecture — how is it different from monolith?

● Hard

ANSWER

Aspect	Monolith CI/CD	Microservices CI/CD
Pipeline count	One pipeline	One pipeline per service
Build	Build entire app	Build only changed service
Test scope	Full test suite	Service tests + contract tests
Deploy unit	Whole application	Individual service
Rollback	Entire application	Individual service rollback
Coordination	Single pipeline	Orchestration across services needed
Speed	Slow — full rebuild each time	Fast — only changed services pipeline
Complexity	Low — one pipeline to maintain	High — N pipelines, versioning, contracts

Key microservices CI/CD challenges:

- ▶ Contract testing — use Pact or Spring Cloud Contract to verify API compatibility between services
- ▶ Service versioning — semantic versioning + backward compatibility windows
- ▶ Deployment ordering — services with dependencies must deploy in correct order
- ▶ Environment parity — each service needs its own staging environment or shared namespace

KEY TAKEAWAYS

- ▶ Microservices = one pipeline per service — independent deployability is the whole point
- ▶ Contract testing = validate API compatibility before services are deployed together
- ▶ Shared staging namespace per PR is common — avoids maintaining N staging environments

SECTION 09

Real-World Scenarios & Troubleshooting

End-to-end scenarios, debugging, pipeline design decisions

Q53

Your production deployment pipeline succeeded but the application is broken — walk through your response.

● Hard

ANSWER

This is a post-deployment production incident. The pipeline passing does not guarantee the application is healthy — smoke tests, synthetic monitoring, and health checks exist for exactly this scenario.

Immediate response (first 5 minutes):

- Step 1: Assess blast radius — how many users affected? Is it total outage or partial?
- Step 2: Rollback immediately if possible — don't debug in production under load
 - Helm: `helm rollback myapp 0`
 - kubectl: `kubectl rollout undo deployment/myapp`
 - ArgoCD: sync to previous Git commit

- Step 3: Verify rollback restored service — check health endpoint, error rate

- Step 4: Communicate — notify stakeholders, update status page

Post-incident (after service restored):

- Step 5: Collect evidence — logs, metrics, traces from the failure window
- Step 6: Identify root cause — what did the pipeline NOT catch?
- Step 7: Add missing test/check to pipeline — prevent recurrence
- Step 8: Blameless post-mortem — document timeline, root cause, action items

📌 'Rollback first, debug later' is the senior DevOps answer. Never debug in production under active incident pressure. Restore service first. The evidence (logs, metrics) will still be there after rollback.

KEY TAKEAWAYS

- Rollback first — restore service before investigating root cause
- Pipeline passing ≠ application healthy — smoke tests and synthetic monitoring catch this gap
- Post-mortem: every production incident should add a test to prevent recurrence

Q54

Design a complete CI/CD pipeline for a microservices app on Kubernetes — tools, stages, and decisions.

● Hard

ANSWER

Complete pipeline design for a Node.js microservices app on Kubernetes:

Stage	Tool	Action	Gate
-------	------	--------	------

Source	GitHub	PR triggers pipeline; branch protection on main	PR required reviewers
Lint + Unit Test	GitHub Actions + Jest	Run in parallel; coverage report	Coverage > 80%
SAST + SCA	Semgrep + Snyk	Static security + dependency scan	No critical CVEs
Build	Docker + BuildKit	Multi-stage build; tag with SHA	Must succeed
Container Scan	Trivy	Scan image for CVEs before push	No HIGH/CRITICAL
Push	Amazon ECR	Push immutable image SHA tag	Must succeed
Deploy Staging	ArgoCD	Update image tag in Git; ArgoCD syncs	Auto
Integration Tests	Postman/Newman	API tests against staging	All tests pass
E2E Tests	Playwright	User journey tests on staging	All tests pass
Deploy Prod	ArgoCD	Manual approval gate; update prod Git	Named approver
Smoke Tests	Synthetic monitoring	Verify prod endpoints post-deploy	Pass within 5min
Monitor	Prometheus + Grafana	Error rate, p99 latency alerts	Ongoing

Key decisions: GitHub Actions for CI (native GitHub integration); ArgoCD for CD (GitOps, no cluster credentials in CI); ECR for registry (AWS native); Helm for packaging; separate app and config repos.

KEY TAKEAWAYS

- ▶ CI (GitHub Actions) = build + test + scan + push; CD (ArgoCD) = deploy — separated responsibilities
- ▶ Every stage is a gate — nothing reaches staging that failed unit tests, SAST, or container scan
- ▶ GitOps = CI updates Git, ArgoCD deploys — CI never touches the cluster directly

Q55**How do you implement zero-downtime deployments in a CI/CD pipeline?**● **Hard****ANSWER**

Zero-downtime deployment requires coordination between the pipeline, Kubernetes, application code, and load balancer. Multiple components must work together.

Component	Configuration	Purpose
Kubernetes readiness probe	httpGet /health, initialDelaySeconds: 10	Pod only receives traffic when truly ready
Rolling update strategy	maxUnavailable: 0, maxSurge: 1	Never take a pod down before new one is ready

Graceful shutdown

SIGTERM handler,
terminationGracePeriodSeconds: 30Finish in-flight
requests before
pod stops

PodDisruptionBudget

minAvailable: 50%

Prevent too many
pods being
removed at once

Helm --wait flag

--wait --timeout 5m

Pipeline waits for
all pods healthy
before declaring
success

Backward-compatible migrations

Expand-contract
patternDB schema works
with old and new
code
simultaneously

```
// Graceful shutdown handler (Node.js)
process.on('SIGTERM', async () => {
  console.log('SIGTERM received - graceful shutdown starting');
  // Stop accepting new connections
  server.close(() => {
    console.log('HTTP server closed');
    // Close DB connections
    await db.end();
    process.exit(0);
  });
  // Force exit after 25s (before K8s kills at 30s)
  setTimeout(() => process.exit(1), 25000);
});
```

KEY TAKEAWAYS

- ▶ Zero downtime = readiness probe + maxUnavailable:0 + graceful shutdown + backward-compatible migrations
- ▶ Readiness probe is the most critical — without it, traffic hits pods before they're ready
- ▶ terminationGracePeriodSeconds must be longer than your graceful shutdown timeout

Q56**What is pipeline observability and how do you monitor CI/CD pipeline health?**

● Medium

ANSWER

Pipeline observability means collecting metrics, logs, and traces from the CI/CD pipeline itself — treating the pipeline as a production system that needs monitoring.

Metric	What to Measure	Why It Matters
Deployment Frequency	Deployments per day/week	DORA metric — higher = more mature
Lead Time for Changes	Commit to production time	DORA metric — lower = faster delivery
Change Failure Rate	% of deployments causing incidents	DORA metric — lower = higher quality
MTTR	Mean Time to Restore after incident	DORA metric — lower = better recovery

Pipeline Duration	Minutes per stage per pipeline	Identify slow stages to optimize
Pipeline Success Rate	% of builds that pass	Flaky tests, environment issues
Queue Time	Time waiting for a runner	Agent capacity planning

Tools: Jenkins with Prometheus plugin, GitHub Actions with Datadog CI Visibility, GitLab Analytics, custom dashboards in Grafana. The four DORA metrics (Deployment Frequency, Lead Time, Change Failure Rate, MTTR) are the industry standard for measuring CI/CD maturity.

KEY TAKEAWAYS

- ▶ Four DORA metrics = industry standard for CI/CD maturity measurement
- ▶ Track pipeline duration per stage — identify bottlenecks to optimize
- ▶ Change Failure Rate > 15% = too many bugs reaching production; < 5% = elite performance

Q57**How do you handle pipeline secrets rotation without breaking running pipelines?**

● Hard

ANSWER

Secret rotation is critical for security but must be handled carefully to avoid breaking active pipelines and deployments. Safe rotation strategy:

- ▶ Step 1: Generate new secret alongside old one — both valid simultaneously
- ▶ Step 2: Add new secret to CI/CD credentials store (Jenkins/GitHub/GitLab)
- ▶ Step 3: Update pipeline to use new secret reference — test in non-production pipeline first
- ▶ Step 4: Deploy and verify pipelines work with new secret
- ▶ Step 5: Revoke old secret — only after confirming new one works everywhere
- ▶ Step 6: Update documentation/runbooks

```
// Dual-secret transition in GitHub Actions
# Phase 1: Add new secret (NEW_DB_PASSWORD) alongside old (DB_PASSWORD)
# Both secrets exist simultaneously

# Phase 2: Update pipeline to use new secret
- name: Deploy
  env:
    DB_PASSWORD: ${ secrets.NEW_DB_PASSWORD } # switched from DB_PASSWORD
  run: ./deploy.sh

# Phase 3: Delete old secret from GitHub Settings → Secrets
# Phase 4: Rename NEW_DB_PASSWORD → DB_PASSWORD for cleanliness
```

For automated rotation: HashiCorp Vault with dynamic secrets generates a new credential per pipeline run — each run gets a short-lived credential, eliminating manual rotation entirely.

KEY TAKEAWAYS

- ▶ Dual-secret window: add new secret before deleting old — no downtime during rotation
- ▶ Never delete old secret until all pipelines confirmed using new one
- ▶ HashiCorp Vault dynamic secrets = automatic rotation per pipeline run — eliminates manual rotation

Q58

What is trunk-based development and how does it affect CI/CD pipeline design?

● Medium

ANSWER

Trunk-based development (TBD) is a source control branching model where all developers commit directly to 'main' (trunk) or use very short-lived feature branches (< 1 day). It is the foundation of high-frequency CI/CD.

Aspect	Trunk-Based Development	GitFlow
Branch lifetime	Hours to 1 day max	Days to weeks
Integration frequency	Multiple times per day	Weekly/sprint-end
Integration risk	Low — small changes	High — big merge conflicts
CI/CD compatibility	Native — always deployable	Complex — multiple branch pipelines
Feature flags	Required — hide incomplete features	Not needed (long branches hide them)
Release cadence	Multiple per day possible	Sprint or scheduled releases
Suitable for	High-performing teams, SaaS	Traditional enterprises, versioned products

In CI/CD pipeline design for TBD: every commit to main triggers the full pipeline including staging deployment. Feature flags control what's visible in production — code can be deployed but features hidden until ready.

KEY TAKEAWAYS

- ▶ Trunk-based = all commits to main; short-lived feature branches (< 1 day)
- ▶ Requires feature flags to hide incomplete features in production
- ▶ TBD + full pipeline on every commit = true continuous integration — the highest CI/CD maturity

Q59

How do you implement feature flags in a CI/CD pipeline?

● Medium

ANSWER

Feature flags (feature toggles) allow code to be deployed to production but features to remain hidden or accessible only to specific users. They decouple deployment from release.

Feature flag use cases in CI/CD:

- ▶ Deploy incomplete features hidden from users
- ▶ A/B test new features with % of users
- ▶ Kill switch — instantly disable a broken feature without rollback
- ▶ Canary feature releases — enable for internal users first, then gradually roll out

Flag Type	Example	Risk Level
Release toggle	Hide incomplete feature until ready	Low — temporary
Experiment toggle	A/B test — 50% users get new UI	Medium — affects UX
Ops toggle	Kill switch for expensive feature under load	Medium — business impact
Permission toggle	Beta feature for specific users/orgs	Low — controlled rollout

Tools: LaunchDarkly, Unleash, Flagsmith, AWS AppConfig, Azure App Configuration. Feature flags should be short-lived — long-lived flags become technical debt.

KEY TAKEAWAYS

- ▶ Feature flags = deploy code + hide features = decouple deployment from release
- ▶ Kill switch = instant feature disable without pipeline rollback
- ▶ Remove feature flags after feature is stable — long-lived flags are tech debt

Q60

End-to-end scenario: A startup wants to set up CI/CD from scratch. Walk through the complete toolchain decision, pipeline design, and implementation plan.

● **Hard**

ANSWER

Complete CI/CD setup for a startup with a Node.js API + React frontend on Kubernetes (AWS EKS):

Tool decisions:

Category	Tool Choice	Reason
Source Control	GitHub	Industry standard; Actions integration; PR workflows
CI Platform	GitHub Actions	Zero server management; free tier generous; native GitHub
Container Registry	Amazon ECR	AWS-native; free within AWS network; IAM integration
Kubernetes	Amazon EKS	Managed control plane; AWS networking; IRSA for pod IAM
CD/GitOps	ArgoCD	Best UI; strong community; Helm native support
Secrets	AWS Secrets Manager + ESO	AWS-native; auto-sync to K8s secrets
Monitoring	Prometheus + Grafana	Open source; K8s native; cost-effective
Code Quality	SonarCloud	SaaS; GitHub PR integration; free for open source

Implementation phases:

- ▶ Week 1: GitHub repo structure, branch protection, PR templates, Actions basic build+test
- ▶ Week 2: Docker multi-stage build, ECR push, Trivy scan, SonarCloud integration
- ▶ Week 3: ArgoCD setup on EKS, Helm charts for each service, GitOps app+config repo separation
- ▶ Week 4: Staging environment ArgoCD app, integration tests in pipeline, manual prod gate
- ▶ Week 5: Prometheus/Grafana, DORA metrics dashboard, runbooks, on-call setup

🔗 This question tests architectural thinking. Show you consider: zero-server-management first (SaaS tools), GitOps for K8s deployments, secrets management from day 1, and phased implementation — not big-bang.

KEY TAKEAWAYS

- ▶ Start with GitHub Actions — zero infrastructure, generous free tier, grows with the team
- ▶ GitOps (ArgoCD) from day 1 — easier to start right than migrate later
- ▶ Phase implementation: CI first (week 1-2), then CD (week 3-4), then observability (week 5)